



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

APLICACIONES WEB

“Práctica en clase: Proyecto ASP.NET Core MVC”

ESTUDIANTE:

FRED ADRIÁN BELTRÁN MONTIEL

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERON

QUINTO SEMESTRE “A”

PPA 2026-2027

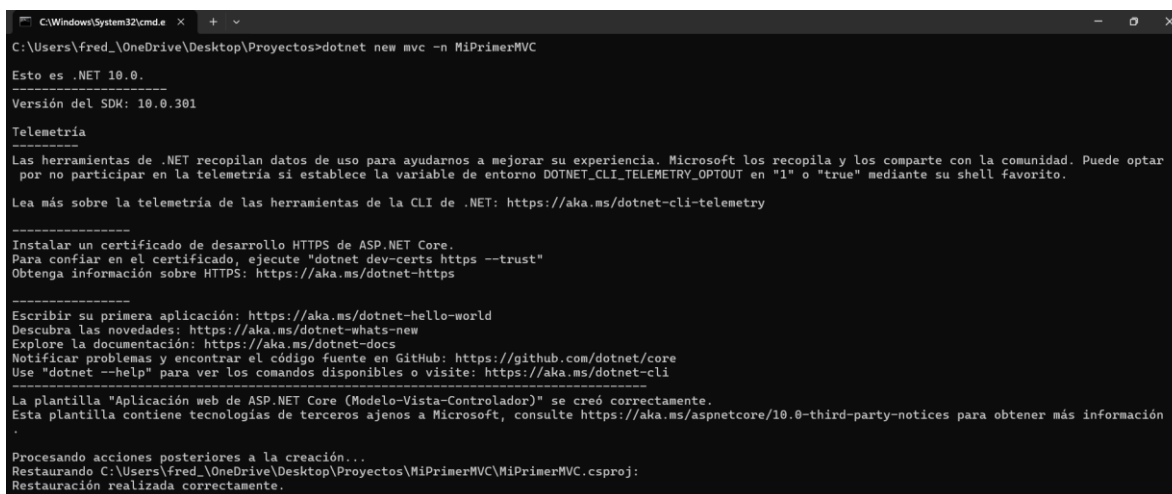
Objetivo

Desarrollar una aplicación web utilizando el patrón **MVC (Modelo-Vista-Controlador)** con **ASP.NET Core**, implementando un controlador con operaciones básicas de listado y creación de datos (Index y Create), un modelo de dominio (Producto) validado mediante **Data Annotations**, y una vista Razor protegida contra ataques de tipo **CSRF (Cross-Site Request Forgery)** mediante el uso de **AntiForgeryToken**. Asimismo, se busca verificar de manera práctica que la validación de datos se ejecute correctamente del lado del servidor (**server-side validation**), garantizando la integridad de la información incluso si las validaciones del lado del cliente fallan o son evadidas.

Paso a paso

1. Creación del proyecto

Se generó un nuevo proyecto ASP.NET Core MVC mediante el comando `dotnet new mvc -n MiPrimerMVC`, el cual creó automáticamente la estructura base de carpetas (Controllers, Models, Views, wwwroot) y los archivos de configuración necesarios (Program.cs, MiPrimerMVC.csproj). Se verificó el correcto funcionamiento del proyecto ejecutando `dotnet run` y accediendo a la página de bienvenida por defecto en el navegador.



```
C:\Windows\System32\cmd.exe
C:\Users\fred_OneDrive\Desktop\Proyectos>dotnet new mvc -n MiPrimerMVC

Esto es .NET 10.0.0.
Versión del SDK: 10.0.301

Telemetría
Las herramientas de .NET recopilan datos de uso para ayudarnos a mejorar su experiencia. Microsoft los recopila y los comparte con la comunidad. Puede optar por no participar en la telemetría si establece la variable de entorno DOTNET_CLI_TELEMETRY_OPTOUT en "1" o "true" mediante su shell favorito.
Lea más sobre la telemetría de las herramientas de la CLI de .NET: https://aka.ms/dotnet-cli-telemetry

-----
Instalar un certificado de desarrollo HTTPS de ASP.NET Core.
Para confiar en el certificado, ejecute "dotnet dev-certs https --trust"
Obtenga información sobre HTTPS: https://aka.ms/dotnet-https

-----
Escribir su primera aplicación: https://aka.ms/dotnet-hello-world
Descubre las novedades: https://aka.ms/dotnet-what's-new
Explore la documentación: https://aka.ms/dotnet-docs
Notificar problemas y encontrar el código fuente en GitHub: https://github.com/dotnet/core
Use "dotnet --help" para ver los comandos disponibles o visite: https://aka.ms/dotnet-cli

-----
La plantilla "Aplicación web de ASP.NET Core (Modelo-Vista-Controlador)" se creó correctamente.
Esta plantilla contiene tecnologías de terceros ajenos a Microsoft, consulte https://aka.ms/aspnetcore/10.0-third-party-notice para obtener más información
.

Procesando acciones posteriores a la creación...
Restaurando C:\Users\fred_OneDrive\Desktop\Proyectos\MiPrimerMVC\MiPrimerMVC.csproj:
Restauración realizada correctamente.
```

Figura 1. Ejecución del comando `dotnet new mvc -n MiPrimerMVC` en la carpeta del proyecto

```

C:\Users\fred_\OneDrive\Desktop\Proyectos>cd MiPrimerMVC

C:\Users\fred_\OneDrive\Desktop\Proyectos\MiPrimerMVC>dir
El volumen de la unidad C es Windows
El número de serie del volumen es: D6FA-5722

Directorio de C:\Users\fred_\OneDrive\Desktop\Proyectos\MiPrimerMVC

17/06/2026  11:02 p. m.    <DIR>          .
17/06/2026  11:02 p. m.    <DIR>          ..
17/06/2026  11:02 p. m.             127 appsettings.Development.json
17/06/2026  11:02 p. m.             151 appsettings.json
17/06/2026  11:02 p. m.    <DIR>          Controllers
17/06/2026  11:02 p. m.             220 MiPrimerMVC.csproj
17/06/2026  11:02 p. m.    <DIR>          Models
17/06/2026  11:02 p. m.    <DIR>          obj
17/06/2026  11:02 p. m.             698 Program.cs
17/06/2026  11:02 p. m.    <DIR>          Properties
17/06/2026  11:02 p. m.    <DIR>          Views
17/06/2026  11:02 p. m.    <DIR>          wwwroot
                        4 archivos             1,196 bytes
                        8 dirs  10,980,712,448 bytes libres

```

Figura 2. Ejecución del comando cd MiPrimerMVC para comprobar la estructura del proyecto

```

C:\Users\fred_\OneDrive\Desktop\Proyectos\MiPrimerMVC>dotnet run
Usando la configuración de inicio de C:\Users\fred_\OneDrive\Desktop\Proyectos\MiPrimerMVC\Properties\launchSettings.json...
Compilando...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5089
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\fred_\OneDrive\Desktop\Proyectos\MiPrimerMVC

```

Figura 3. Ejecución del comando dotnet run para comprobar si el localhost funciona correctamente

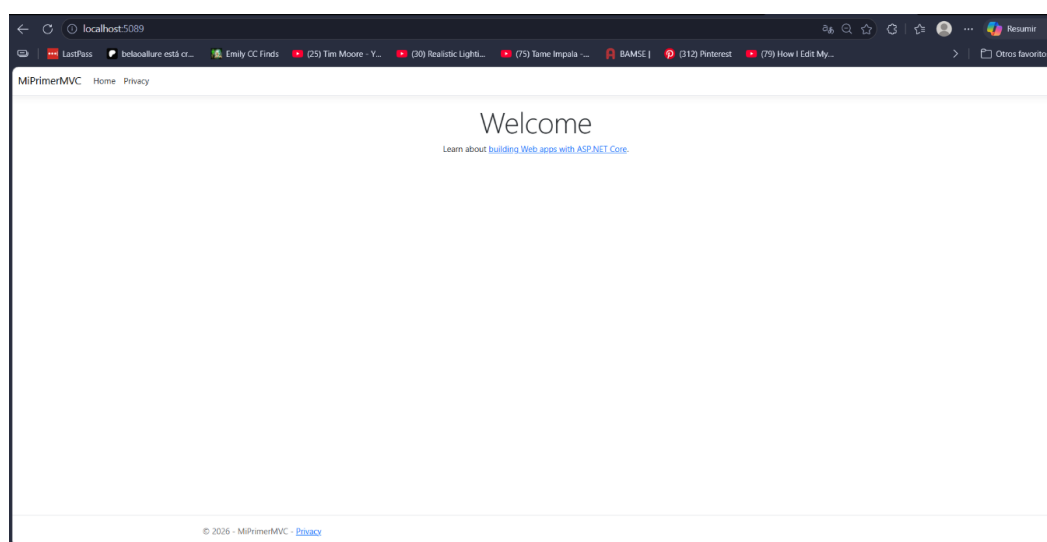


Figura 4. Evidencia del localhost funcionando

2. Creación del modelo Producto

Dentro de la carpeta Models, se creó la clase Producto.cs, la cual define las propiedades Id, Nombre, Descripción, Precio y Stock. Sobre estas propiedades se aplicaron **Data Annotations** ([Required], [StringLength], [Range], [DataType]) para establecer reglas de validación: campos obligatorios, longitud máxima de texto, y rangos numéricos válidos para precio y stock.

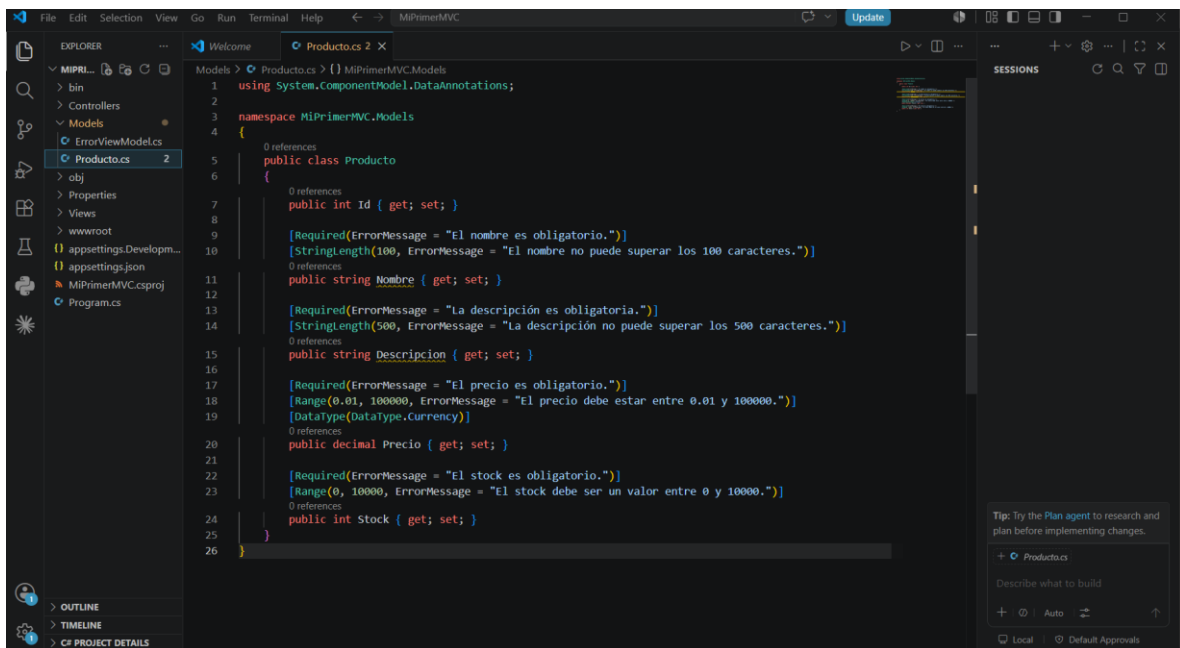


Figura 5. Captura del Producto.cs en VS CODE

Producto.cs

```
using System.ComponentModel.DataAnnotations;

namespace MiPrimerMVC.Models
{
    public class Producto
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "El nombre es obligatorio.")]
        [StringLength(100, ErrorMessage = "El nombre no puede superar los 100 caracteres.")]
        public string Nombre { get; set; }

        [Required(ErrorMessage = "La descripción es obligatoria.")]
        [StringLength(500, ErrorMessage = "La descripción no puede superar los 500 caracteres.")]
        public string Descripcion { get; set; }
    }
}
```

```

        [Required(ErrorMessage = "El precio es obligatorio.")]
        [Range(0.01, 100000, ErrorMessage = "El precio debe estar entre 0.01
y 100000.")]
        [DataType(DataType.Currency)]
        public decimal Precio { get; set; }

        [Required(ErrorMessage = "El stock es obligatorio.")]
        [Range(0, 10000, ErrorMessage = "El stock debe ser un valor entre 0
y 10000.")]
        public int Stock { get; set; }
    }
}

```

3. Creación del controlador ProductoController

Se implementó la clase ProductoController.cs con dos acciones principales:

- Index(): retorna la lista de productos registrados (almacenados en memoria para esta práctica).
- Create(): cuenta con dos versiones, una GET, que muestra el formulario vacío, y una POST, que recibe los datos enviados, valida el modelo mediante ModelState.IsValid y, si es correcto, agrega el producto a la lista y redirige al Index.

La acción Create de tipo POST se decoró con el atributo [ValidateAntiForgeryToken], exigiendo que la solicitud incluya un token antifalsificación válido.

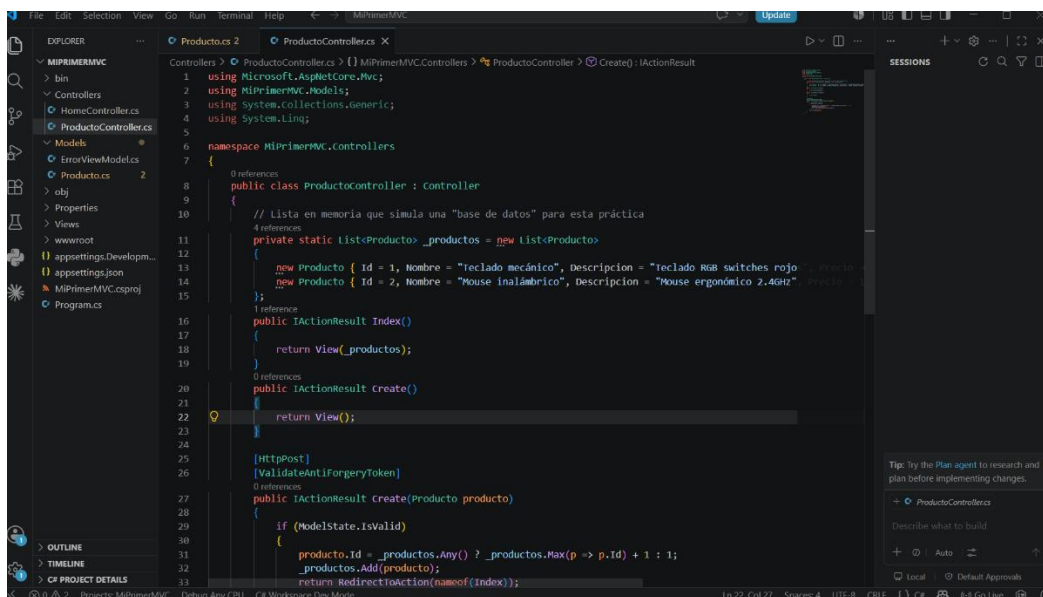


Figura 6. Captura del ProductoController.cs en VS Code

ProductController.cs

```
using Microsoft.AspNetCore.Mvc;
using MiPrimerMVC.Models;
using System.Collections.Generic;
using System.Linq;

namespace MiPrimerMVC.Controllers
{
    public class ProductoController : Controller
    {
        private static List<Producto> _productos = new List<Producto>
        {
            new Producto { Id = 1, Nombre = "Teclado mecánico", Descripcion = "Teclado RGB switches rojos", Precio = 45.99m, Stock = 20 },
            new Producto { Id = 2, Nombre = "Mouse inalámbrico", Descripcion = "Mouse ergonómico 2.4GHz", Precio = 19.99m, Stock = 35 }
        };

        public IActionResult Index()
        {
            return View(_productos);
        }

        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Create(Producto producto)
        {
            if (ModelState.IsValid)
            {
                producto.Id = _productos.Any() ? _productos.Max(p => p.Id) + 1 : 1;
                _productos.Add(producto);
                return RedirectToAction(nameof(Index));
            }
            return View(producto);
        }
    }
}
```

4. Creación de las vistas Razor

Se generaron las vistas Index.cshtml y Create.cshtml dentro de Views/Producto. La vista Index recorre la lista de productos y los muestra en una tabla HTML. La vista Create contiene un formulario con un campo por cada propiedad del modelo, e incluye la directiva `@Html.AntiForgeryToken()`, la cual genera un campo oculto con un token único de seguridad que el servidor valida al recibir el formulario, evitando que terceros no autorizados puedan enviar solicitudes falsificadas al servidor.

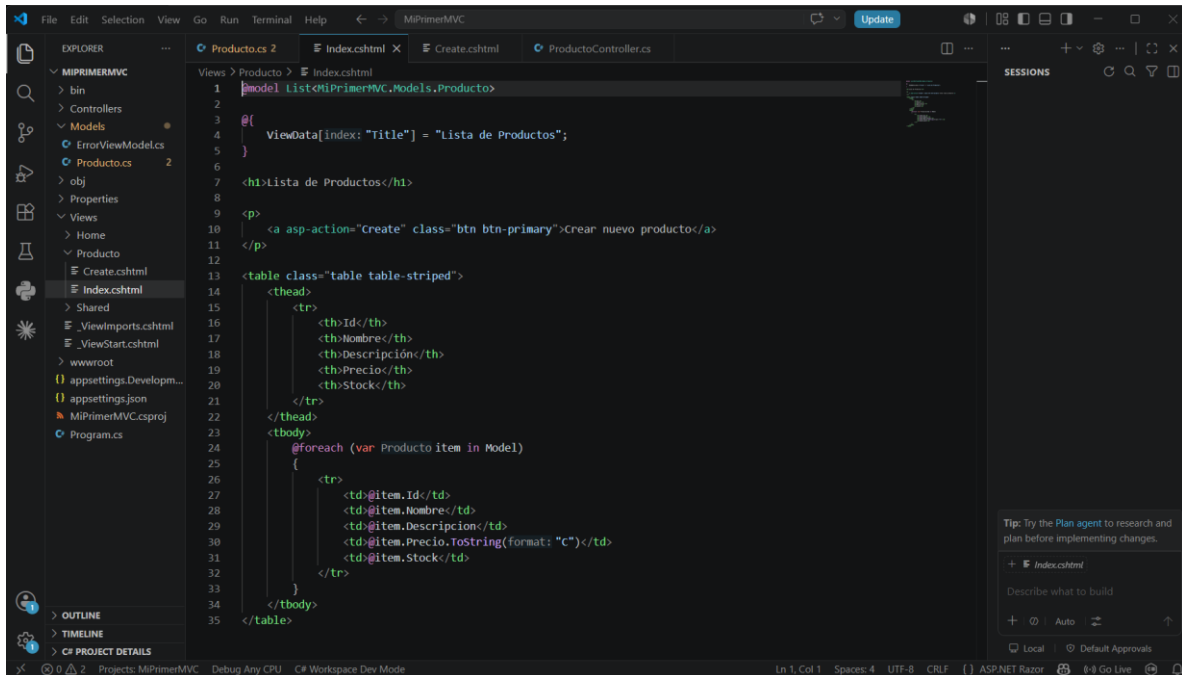


Figura 7. Captura del Index.cshtml en VS Code

Index.cshtml

```
@model List<MiPrimerMVC.Models.Producto>

@{
    ViewData["Title"] = "Lista de Productos";
}

<h1>Lista de Productos</h1>

<p>
    <a asp-action="Create" class="btn btn-primary">Crear nuevo producto</a>
</p>

<table class="table table-striped">
    <thead>
        <tr>
```

```

        <th>Id</th>
        <th>Nombre</th>
        <th>Descripción</th>
        <th>Precio</th>
        <th>Stock</th>
    </tr>
</thead>
<tbody>
    @foreach (var item in Model)
    {
        <tr>
            <td>@item.Id</td>
            <td>@item.Nombre</td>
            <td>@item.Descripcion</td>
            <td>@item.Precio.ToString("C")</td>
            <td>@item.Stock</td>
        </tr>
    }
</tbody>
</table>

```

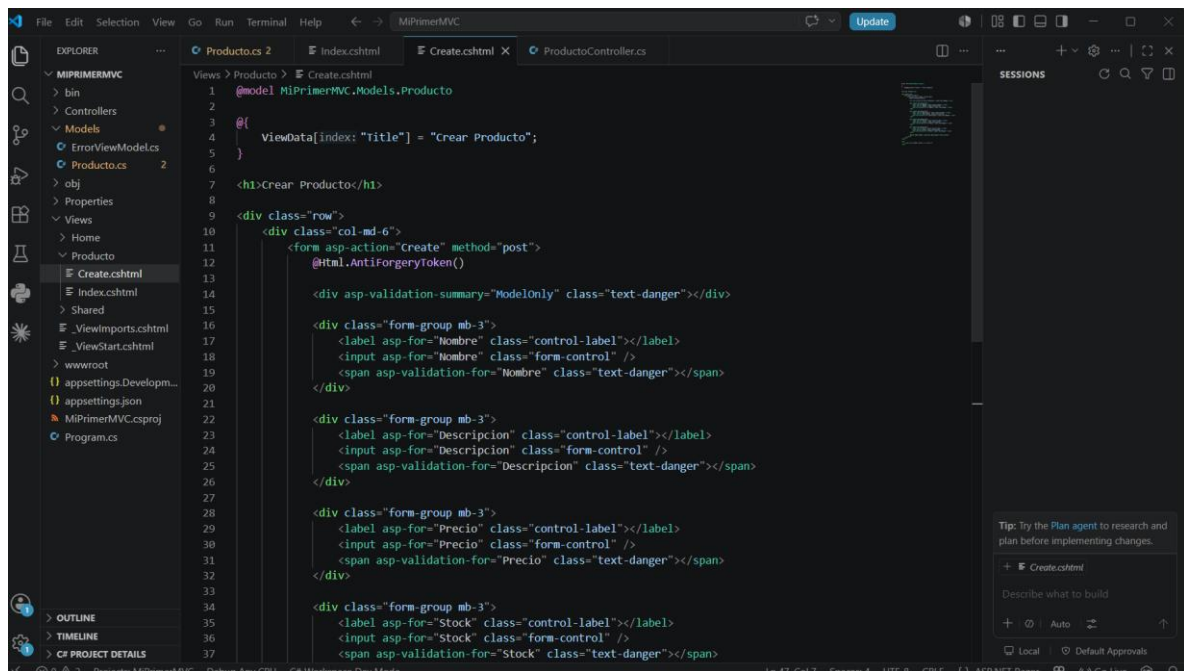


Figura 8. Captura de Create.cshtml en VS Code

Create.cshtml

```
@model MiPrimerMVC.Models.Producto
```



```

@{
    ViewData["Title"] = "Crear Producto";
}

<h1>Crear Producto</h1>

<div class="row">
    <div class="col-md-6">
        <form asp-action="Create" method="post">
            @Html.AntiForgeryToken()

            <div asp-validation-summary="ModelOnly" class="text-
danger"></div>

            <div class="form-group mb-3">
                <label asp-for="Nombre" class="control-label"></label>
                <input asp-for="Nombre" class="form-control" />
                <span asp-validation-for="Nombre" class="text-
danger"></span>
            </div>

            <div class="form-group mb-3">
                <label asp-for="Descripcion" class="control-label"></label>
                <input asp-for="Descripcion" class="form-control" />
                <span asp-validation-for="Descripcion" class="text-
danger"></span>
            </div>

            <div class="form-group mb-3">
                <label asp-for="Precio" class="control-label"></label>
                <input asp-for="Precio" class="form-control" />
                <span asp-validation-for="Precio" class="text-
danger"></span>
            </div>

            <div class="form-group mb-3">
                <label asp-for="Stock" class="control-label"></label>
                <input asp-for="Stock" class="form-control" />
                <span asp-validation-for="Stock" class="text-danger"></span>
            </div>

            <button type="submit" class="btn btn-primary">Crear</button>
        </form>
    </div>
</div>

```

```
<div>
  <a asp-action="Index">Volver a la lista</a>
</div>
```

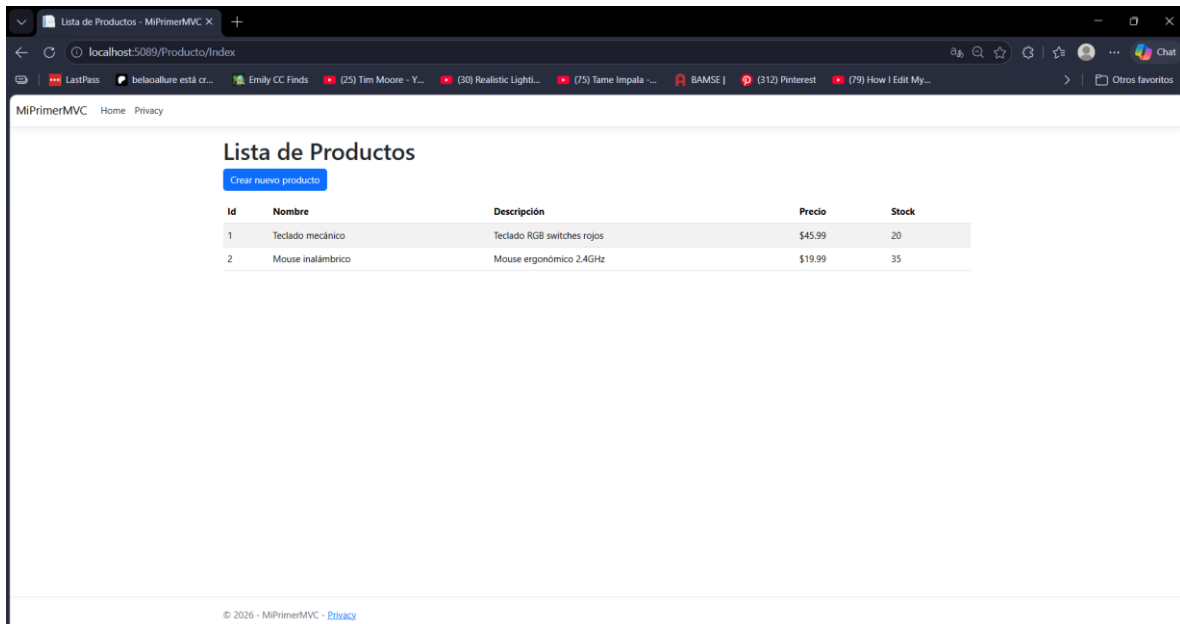


Figura 9. Captura del Index en el navegador funcionando correctamente

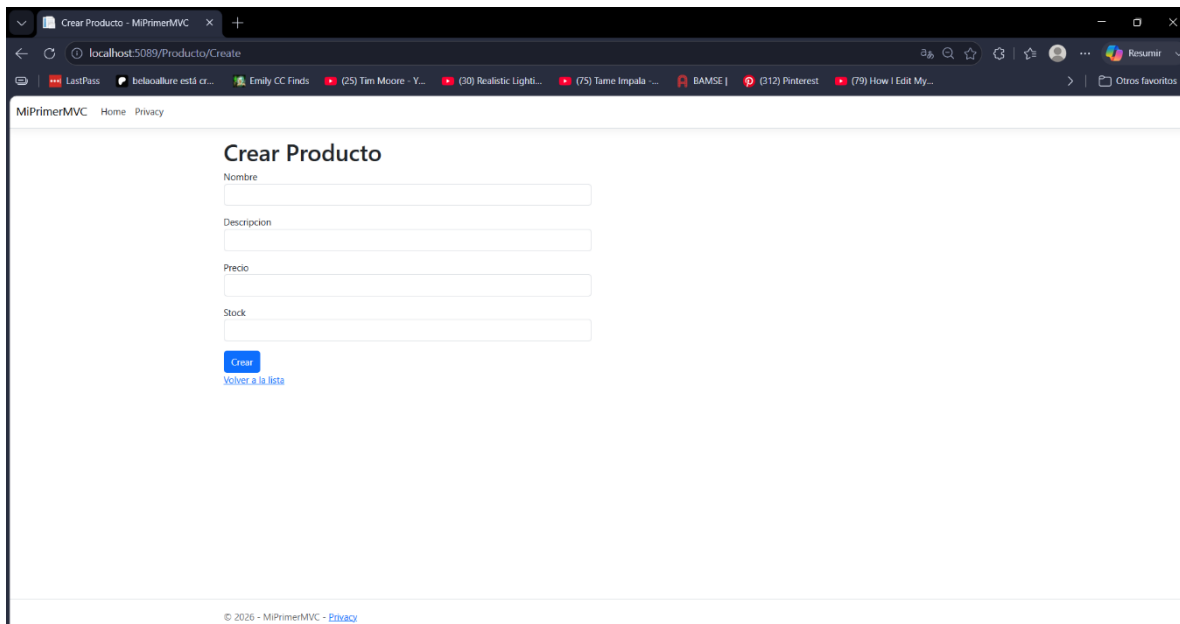


Figura 10. Captura del formulario vacío

5. Verificación de la validación server-side

Se ejecutó el proyecto y se accedió al formulario de creación. Al enviar el formulario con campos vacíos o con valores fuera de rango (por ejemplo, precio negativo), el servidor rechazó la operación y devolvió la misma vista mostrando los mensajes de error correspondientes a cada Data Annotation, sin completar el registro. Al llenar el formulario con datos válidos, el servidor procesó correctamente la solicitud, agregó el nuevo producto a la lista y redirigió a la vista Index, donde se evidenció el nuevo registro. Esto confirmó que la validación del modelo se ejecuta efectivamente en el servidor (`ModelState.IsValid`), de forma independiente a cualquier validación que el navegador realice en el lado del cliente.

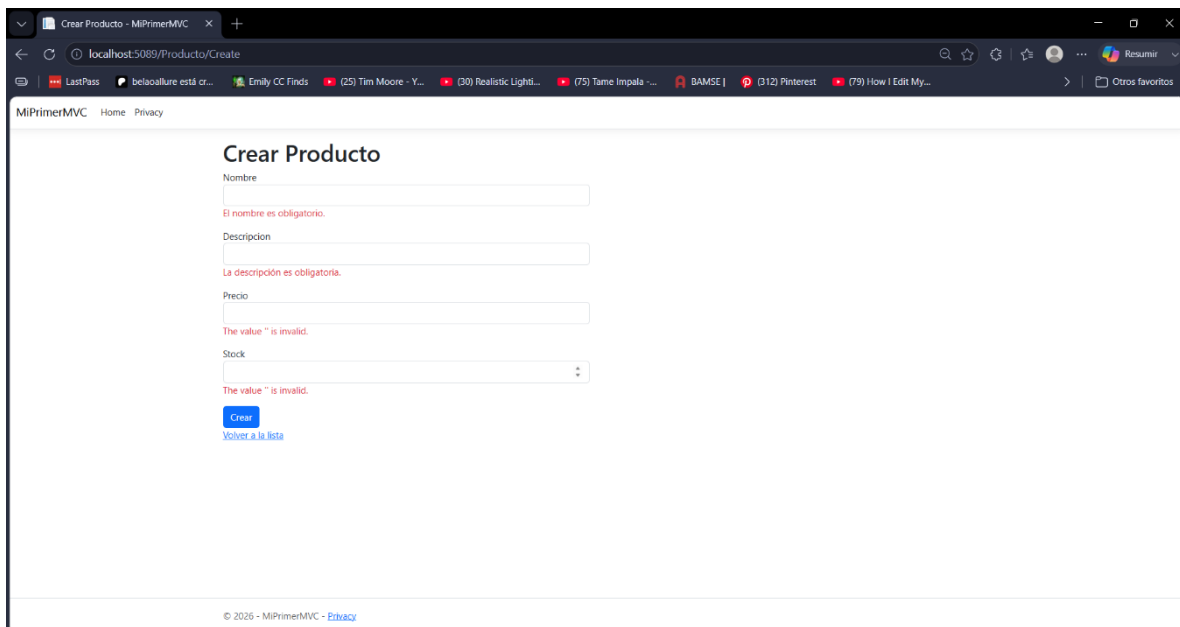


Figura 11. Captura del envío del formulario con campos vacíos

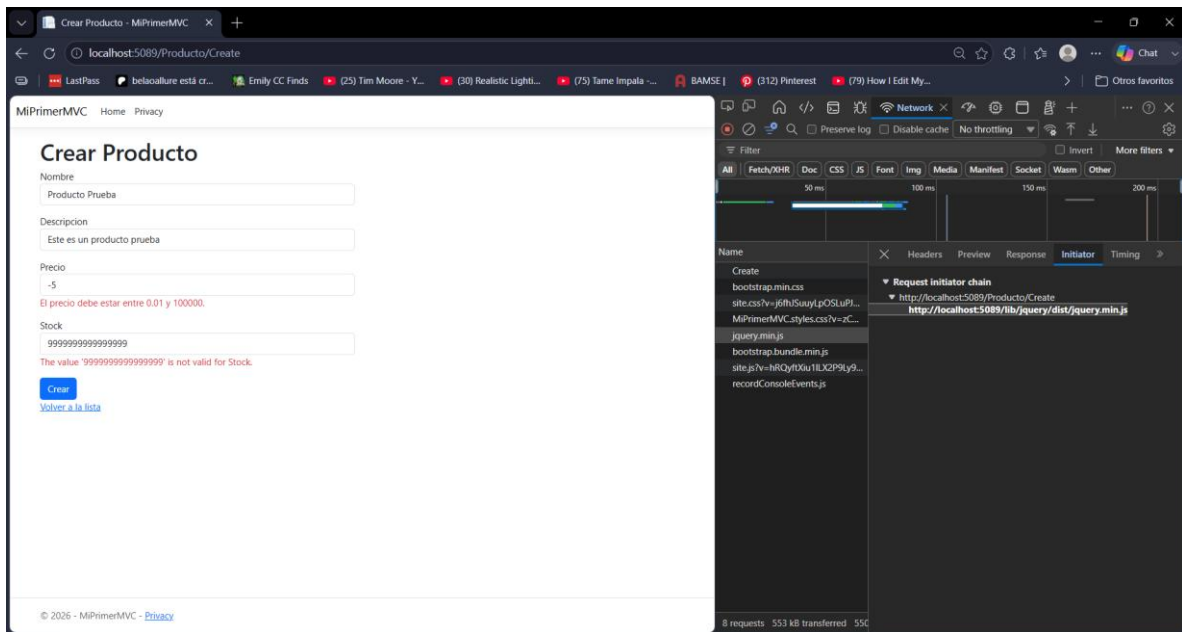


Figura 12. Validaciones para valores negativos y no contemplados en el formulario (Manejo de excepciones correctas con el 200)

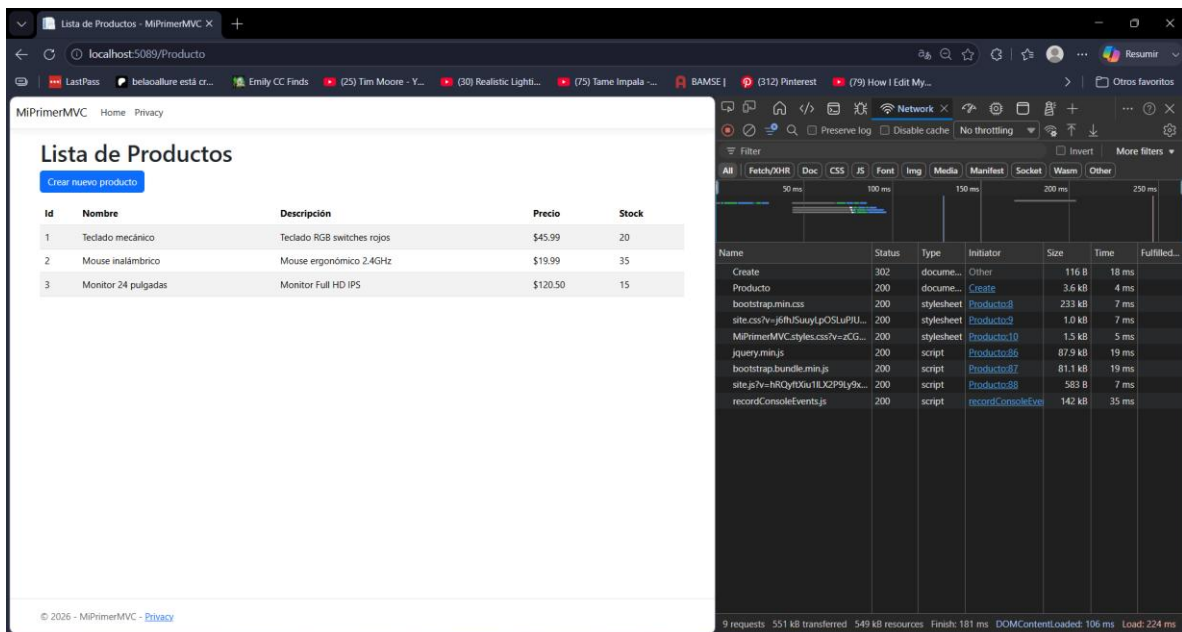


Figura 13. Captura de registro correcto de un nuevo producto, se actualiza el Index correctamente

Conclusión

A través de esta práctica se logró comprender el funcionamiento integral del patrón MVC en ASP.NET Core, identificando el rol específico de cada componente: el **modelo** como portador de las reglas de negocio y validación de datos, el **controlador** como intermediario que gestiona la lógica de las solicitudes HTTP, y la **vista** como la capa encargada de la presentación e interacción con el usuario.

Se evidenció la importancia de las **Data Annotations** como mecanismo declarativo para definir reglas de validación directamente sobre el modelo, evitando la necesidad de escribir lógica de validación repetitiva en el controlador. Asimismo, se comprobó que el uso del **AntiForgeryToken** constituye una medida de seguridad esencial en formularios web, ya que protege a la aplicación frente a ataques CSRF, asegurando que únicamente solicitudes legítimas, originadas desde el propio formulario de la aplicación, sean procesadas por el servidor.

Finalmente, se verificó de forma práctica que la validación **server-side** es indispensable y no puede sustituirse únicamente por validaciones del lado del cliente, ya que estas últimas pueden ser desactivadas o evadidas. Gracias a la propiedad `ModelState.IsValid`, el servidor garantiza que solo se persistan datos que cumplan con las reglas definidas en el modelo, reforzando así la integridad y consistencia de la información dentro de la aplicación.